School of Computation, Information and Technology
Technische Universität München

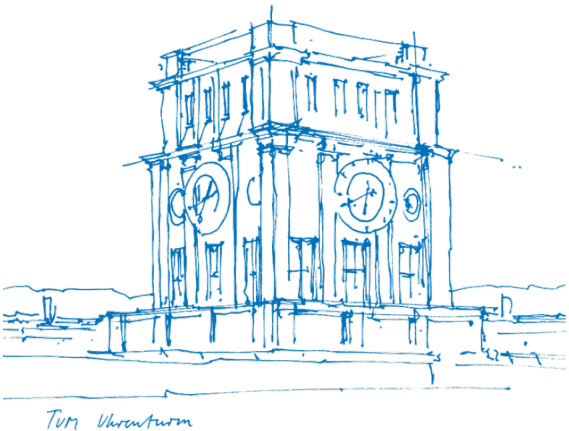
Übung 04: Rekursion und Calling Convention

Einführung in die Rechnerarchitektur

Niklas Ladurner

School of Computation, Information and Technology
Technische Universität München

6. November 2026



Keine Garantie für die Richtigkeit der Tutorfolien.

Bei Unklarheiten/Unstimmigkeiten haben VL/ZÜ-Folien recht!

Caller und Callee

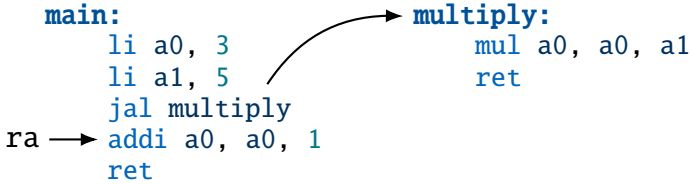
```
main:
    li a0, 3
    li a1, 5
    jal multiply
    addi a0, a0, 1
    ret
```

Caller
(aufrufende Funktion)

```
multiply:
    mul a0, a0, a1
    ret
```

Callee
(aufgerufene Funktion)

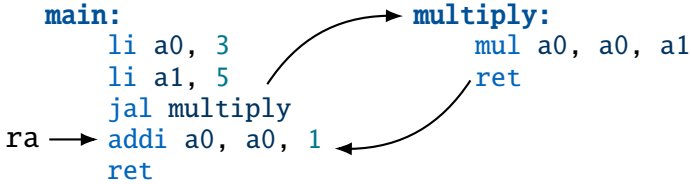
Caller und Callee



Caller
(aufrufende Funktion)

Callee
(aufgerufene Funktion)

Caller und Callee



Caller
(aufrufende Funktion)

Callee
(aufgerufene Funktion)

Register

Register	ABI Name	Description	Saver[1]
x0	zero	Hard-wired zero	—
x1	ra	Return address	Caller
x2	sp	Stack pointer	Callee
x3	gp	Global pointer	—
x4	tp	Thread pointer	—
x5-x7	t0-t2	Temporaries	Caller
x8	s0/fp	Saved register/frame pointer	Callee
x9	s1	Saved register	Callee
x10-11	a0-1	Function arguments/return values	Caller
x12-17	a2-7	Function arguments	Caller
x18-27	s2-11	Saved registers	Callee
x28-31	t3-6	Temporaries	Caller

[1] Erst in zwei Wochen relevant

Calling Convention

- „Aufrufkonvention“ $\rightarrow$ lediglich eine Vereinbarung
- definiert Parameterübergabe, Rückgabe, Registersicherung, Stack etc.
  1. Datentypen $\leq$ 4 Byte in a-Register, sign-extension falls vorzeichenbehafteter Datentyp
  2. Datentypen $=$ 8 Byte in zwei a-Registern, niedrigwertige Hälfte zuerst
  3. Datentypen $>$ 8 Byte als Pointer auf Speicher (meist Caller-Stack)
  4. Falls zu wenige Register: Übergabe über Stack
- Stackpointer muss immer ein Vielfaches von 16 Byte sein!

Calling Convention

```
uint64_t foo(int32_t a, uint8_t b, uint64_t c);
```

Parameter	Übergabe untere 32 Bit	Übergabe obere 32 Bit
a		
b		
c		
Rückgabe		

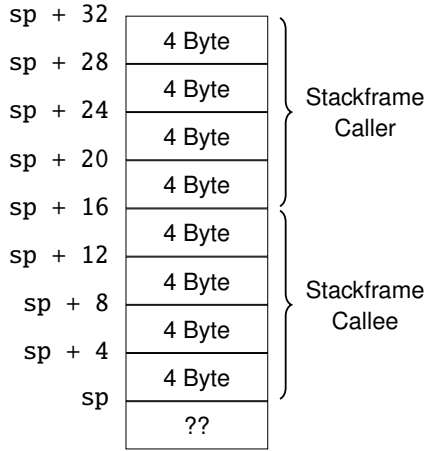
Calling Convention

```
uint64_t foo(int32_t a, uint8_t b, uint64_t c);
```

Parameter	Übergabe untere 32 Bit	Übergabe obere 32 Bit
a	a0	—
b	a1 (zero-extended)	—
c	a2	a3
Rückgabe	a0	a1

Stack

- Speicherbereich für lokale Variablen
- wächst von hohen Adressen zu niedrigen Adressen
- Platz auf dem Stack reservieren: `sp - x`
- Platz auf dem Stack freigeben: `sp + x`
- alles an Adressen < `sp` ungeschützt
- besonders wichtig für Rekursion (Stackframes)



Rekursion (1)

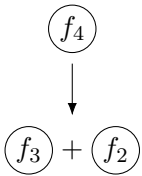
- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$

Rekursion (1)



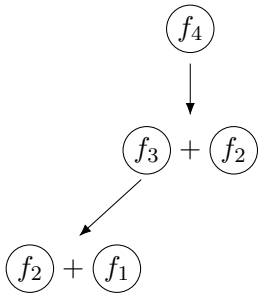
- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$

Rekursion (1)



- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$

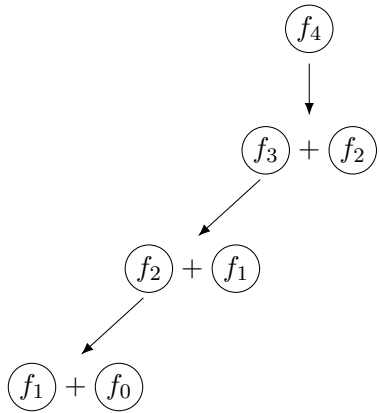
Rekursion (1)



- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$

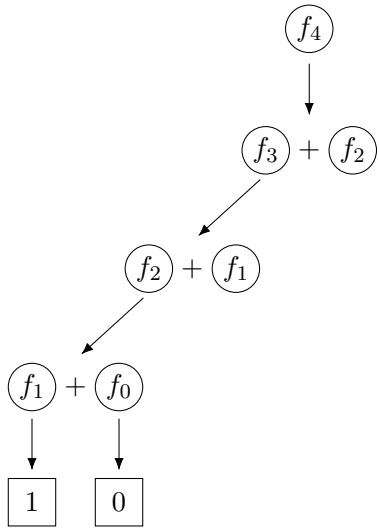
Rekursion (1)

- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$



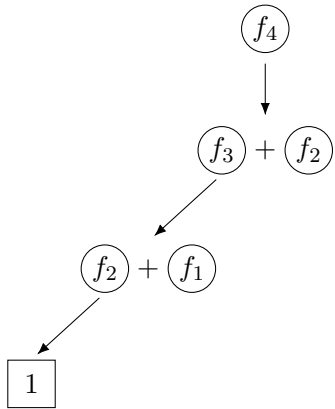
Rekursion (1)

- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$



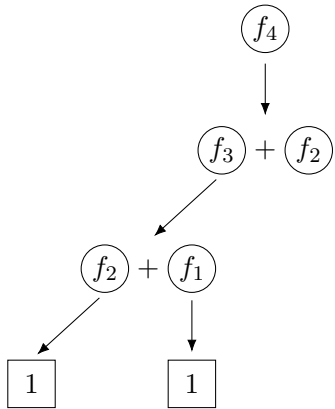
Rekursion (1)

- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$

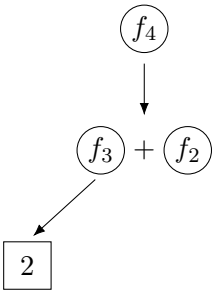


Rekursion (1)

- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$



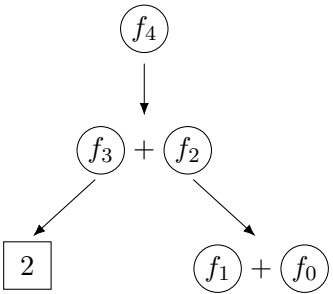
Rekursion (1)



- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$

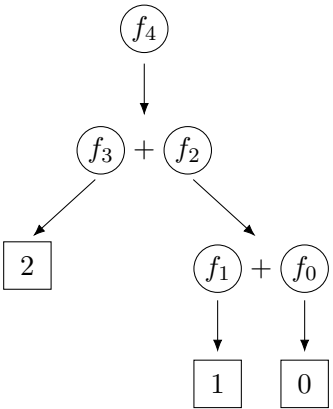
Rekursion (1)

- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$

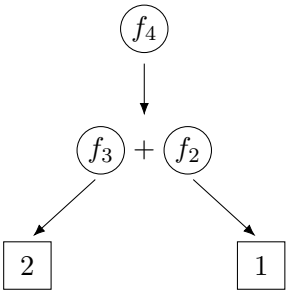


Rekursion (1)

- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$



Rekursion (1)



- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$

Rekursion (1)



- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$

Rekursion (1)

- Funktion die sich selbst aufruft
- Konzept: Berechnung aufteilen bis Basisfall erreicht
- jede rekursive Funktion kann auch iterativ implementiert werden ($\rightarrow$ Schleife)
- Beispiel Fibonacci-Folge:
  $f_n = f_{n-1} + f_{n-2}$,
  $f_1 = 1$, $f_0 = 0$

$$f_4 = 3$$

Rekursion (2)

```
fun:
  addi sp, sp, -8
  sw ra, 0(sp)
  sw a0, 4(sp)
  li t0, 1
  beq a0, t0, end
  addi a0, a0, -1
  jal fun
  lw t0, 4(sp)
  mul a0, a0, t0
end:
  lw ra, 0(sp)
  addi sp, sp, 8
  ret
```

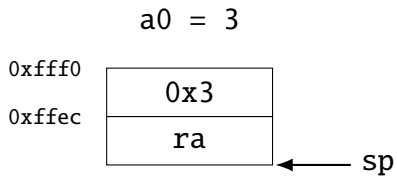
1. Sicherung von `ra` und evtl. Parametern
2. Abbruchbedingung(en)
3. Vorbereitung der Parameter für den rekursiven Aufruf
4. Rekursiver Aufruf
5. Ergebnis des Aufrufs verwerten
6. Wiederherstellung von `ra`, `sp`
7. Rücksprung

Schritte 3–5 können mehrmals vorkommen!

Achtung: 8 Byte nicht CC-konform, nur zur besseren Darstellung

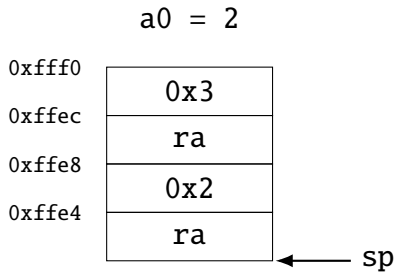
Rekursion (2)

```
fun:
  addi sp, sp, -8
  sw ra, 0(sp)
  sw a0, 4(sp)
  li t0, 1
  beq a0, t0, end
  addi a0, a0, -1
  jal fun
  lw t0, 4(sp)
  mul a0, a0, t0
end:
  lw ra, 0(sp)
  addi sp, sp, 8
  ret
```



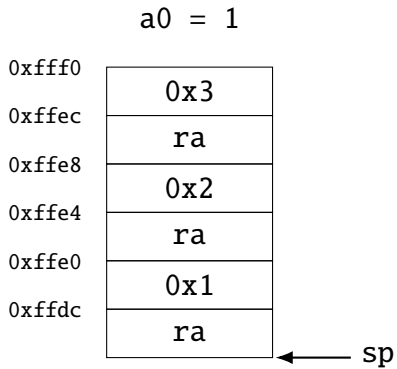
Rekursion (2)

```
fun:
  addi sp, sp, -8
  sw ra, 0(sp)
  sw a0, 4(sp)
  li t0, 1
  beq a0, t0, end
  addi a0, a0, -1
  jal fun
  lw t0, 4(sp)
  mul a0, a0, t0
end:
  lw ra, 0(sp)
  addi sp, sp, 8
  ret
```



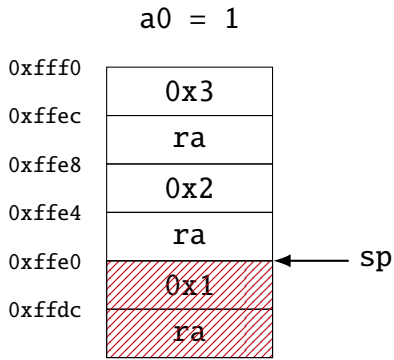
Rekursion (2)

```
fun:
  addi sp, sp, -8
  sw ra, 0(sp)
  sw a0, 4(sp)
  li t0, 1
  beq a0, t0, end
  addi a0, a0, -1
  jal fun
  lw t0, 4(sp)
  mul a0, a0, t0
end:
  lw ra, 0(sp)
  addi sp, sp, 8
  ret
```



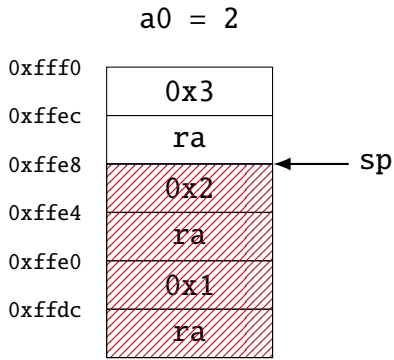
Rekursion (2)

```
fun:
  addi sp, sp, -8
  sw ra, 0(sp)
  sw a0, 4(sp)
  li t0, 1
  beq a0, t0, end
  addi a0, a0, -1
  jal fun
  lw t0, 4(sp)
  mul a0, a0, t0
end:
  lw ra, 0(sp)
  addi sp, sp, 8
  ret
```



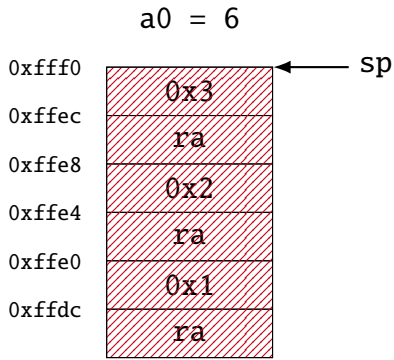
Rekursion (2)

```
fun:
  addi sp, sp, -8
  sw ra, 0(sp)
  sw a0, 4(sp)
  li t0, 1
  beq a0, t0, end
  addi a0, a0, -1
  jal fun
  lw t0, 4(sp)
  mul a0, a0, t0
end:
  lw ra, 0(sp)
  addi sp, sp, 8
  ret
```



Rekursion (2)

```
fun:
  addi sp, sp, -8
  sw ra, 0(sp)
  sw a0, 4(sp)
  li t0, 1
  beq a0, t0, end
  addi a0, a0, -1
  jal fun
  lw t0, 4(sp)
  mul a0, a0, t0
end:
  lw ra, 0(sp)
  addi sp, sp, 8
  ret
```



Fragen?

Links

- Zulip: „ERA Tutorium – Mi-1600-1“ bzw. „ERA Tutorium – Fr-1600-1“
- ERA-Moodle-Kurs
- ERA-Artemis-Kurs
- Übersicht an RISC-V-Instruktionen
- Übersicht an Pseudoinstruktionen

School of Computation, Information and Technology
Technische Universität München

Übung 04: Rekursion und Calling Convention

Einführung in die Rechnerarchitektur

Niklas Ladurner

School of Computation, Information and Technology
Technische Universität München

6. November 2026

